

Universidad Veracruzana

Desarrollo de Sistemas Web (DSW-19559)

P02. Aplicación Web 1.0 con JSP, Tomcat y PostgreSQL

Primer incremento del proyecto PR09

Monitoreo de huerto o ambiente

Equipo DSW-E01

Alejandro Pacheco Luna

Juan Pablo Kuri Ricardez

Ariadna Trejo Álvarez

Pedro García Padilla

Facilitador: Dr. Gabriel Rodríguez Vásquez

Repositorio: github.com/zywoxxx/dsw-p02-monitoreo-huerto

Xalapa, Ver., 11 de septiembre de 2026

Contenido

1. Qué entregamos
 2. Cómo elegimos el proyecto (F02)
 3. Acta breve del proyecto
 4. Modelo de datos
 5. Cómo está construida la aplicación
 6. Instalación y despliegue
 7. Pruebas y evidencia
 8. Problemas que tuvimos y cómo los resolvimos
 9. Relación con la rúbrica R02
 10. Quién hizo qué
 11. Limitaciones y pendientes
- Anexo. Comandos y archivos de referencia

1. Qué entregamos

Este documento acompaña al repositorio del equipo y resume el primer incremento de nuestro proyecto integrador. El proyecto que nos tocó del banco C11 es el PR09, Monitoreo de huerto o ambiente. En esta primera etapa (Web 1.0) construimos una aplicación con JSP y Servlets que corre en Tomcat 9 y guarda todo en PostgreSQL 16. La idea es sencilla: alguien del huerto captura la lectura de un sensor, el sistema la valida, la guarda con su fecha y su procedencia, y si el valor se sale del rango que definimos para ese sensor, genera una alerta. Además se pueden dejar anotaciones por zona.

Todo lo que aparece aquí se puede reproducir con el repositorio: el código, los scripts SQL, un archivo docker-compose para levantar la base de datos, un script que compila, despliega y corre las pruebas de extremo a extremo, y las capturas que se muestran más adelante. Los datos son ficticios. No hay contraseñas reales en ningún archivo versionado.

Elemento	Dónde está
Código JSP/Servlet y configuración	src/ y pom.xml (WAR web1.war)
Modelo de datos y scripts SQL	sql/01_schema.sql, 02_seed.sql, 03_consultas_verificacion.sql, 99_cleanup.sql
README de instalación y despliegue	README.md
Acta breve, selección F02, trazabilidad, bitácora	docs/
Capturas y salidas de consola	docs/evidencia/img/ y docs/evidencia/txt/
Este documento	docs/entrega/P02_EQUIPO_01.docx

2. Cómo elegimos el proyecto (F02)

Antes de programar comparamos tres fichas del banco. Nos interesaba un problema que entendiéramos bien, que se pudiera simular sin hardware y que no nos obligara a resolver autorización por roles desde el primer incremento.

El PR01 (inventario) lo descartamos porque su riesgo principal es la alteración de existencias y la ficha pide bitácora inmutable y permisos por rol; eso nos parecía mucho para arrancar. El PR03 (laboratorio de cómputo) nos gustó porque comparte el patrón de lecturas, umbrales y alertas, pero mezcla inventario de equipos e incidencias y su complejidad orientativa es media-alta. El PR09 tiene el mismo patrón, complejidad media, variables físicas fáciles de explicar (temperatura, humedad, luz) y un riesgo que se controla con una sola decisión de diseño: etiquetar cada lectura con su procedencia para no presentar una simulación como si fuera una medición real.

Por eso pedimos el PR09 como opción principal y dejamos el PR03 como segunda opción. La publicación completa con la plantilla del foro está en docs/F02-seleccion-proyecto.md.

3. Acta breve del proyecto

Campo	Valor
Proyecto	PR09 – Monitoreo de huerto o ambiente (C11)
Equipo	DSW-E01
Actividad	P02, del 31 de agosto al 11 de septiembre de 2026, rúbrica R02
Stack fijo	Java 11, Maven 3.9.9, Tomcat 9.0.115 (Servlet 4.0 / JSP 2.3), PostgreSQL 16, JDBC
Roles funcionales	Responsable de huerto, observador y coordinación (máximo tres, como pide la ficha)

Problema y usuarios

En un huerto escolar las observaciones se anotan en libretas o se mandan por mensaje: no queda claro cuándo se tomó cada dato, en qué unidad ni quién lo midió, y nadie se entera a tiempo cuando algo se sale de rango. Los usuarios son el responsable del huerto (define zonas, sensores y umbrales), el observador (captura lecturas y anotaciones) y coordinación (consulta el historial y las alertas).

Alcance de este incremento

Incluimos zonas, variables con unidad y rango físico, sensores por zona, lecturas con instante y procedencia, umbral por sensor, historial, alertas automáticas y anotaciones por rol, más una ruta de salud para comprobar la conexión a la base. Dejamos fuera lo que la ficha excluye (riego automático, recomendaciones agronómicas, hardware obligatorio) y lo que corresponde a incrementos posteriores: autenticación, alta de zonas y sensores desde pantalla, atención de alertas, la API (RF08) y el simulador IoT (RF09).

Flujo principal

El observador entra a /web1/lecturas y ve los sensores agrupados por zona con su variable y su umbral, y debajo el historial. Elige un sensor, escribe el valor y, si quiere, una observación. Al enviar, el servidor revisa que el sensor exista y esté activo, que el valor sea numérico con máximo dos decimales y que esté dentro del rango físico de la variable. Si algo falla, responde 400 y muestra la lista de errores sin perder lo que ya se había escrito. Si todo está bien, guarda la lectura marcada como manual; si el valor queda fuera del umbral del sensor crea también una alerta (BAJA o ALTA) en la misma transacción, y redirige con 303 a la lista, donde aparece el mensaje de confirmación y la fila nueva.

Requisitos funcionales de la ficha y cómo los cubrimos

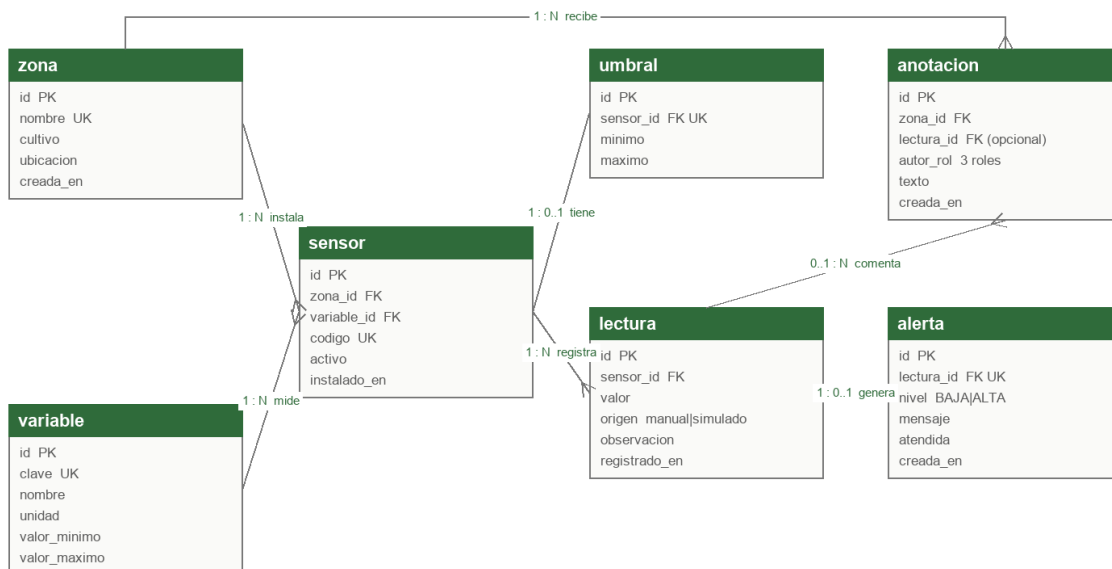
RF	Ficha PR09	Qué hicimos en P02	Cómo se comprueba
RF01	Zonas	Tabla zona; se listan en /anotaciones y aparecen en el catálogo de sensores	Figuras 2 y 10
RF02	Cultivos o variables	Tabla variable con unidad y rango físico; cada sensor muestra la suya	Figura 2
RF03	Lecturas	Formulario POST /lecturas con validación y	Figuras 3, 4 y 7 a 9

		procedencia manual	
RF04	Umbral	Tabla umbral (uno por sensor), visible en el catálogo	Figura 2
RF05	Historial	Lista ordenada por instante; la lectura nueva aparece tras el POST	Figura 4 y sección 7.6
RF06	Alertas y anotaciones	Alerta automática por umbral; página /anotaciones con rol funcional	Figuras 5, 6 y 10 a 12
RF07	Modelo relacionado	7 tablas con llaves foráneas, UNIQUE y CHECK	Secciones 4 y 7.6
RF08 / RF09	API y IoT	No aplican en Web 1.0; la columna origen ya distingue manual/simulado	—

De los criterios de aceptación de la ficha, en este incremento cubrimos CA01 (flujo completo con cambio persistente), CA02 (caso inválido rechazado sin alterar datos), CA05 (otra persona puede reproducirlo con el README) y CA06 (autoría y contribución). CA03 y CA04 corresponden a la API y al evento IoT de etapas posteriores.

4. Modelo de datos

La ficha propone seis entidades: zona, variable, lectura, umbral, alerta y anotación. Nosotros agregamos sensor, porque en la práctica una lectura no la produce una variable abstracta sino un dispositivo instalado en una zona, y ese dispositivo es justo lo que la ficha pide identificar cuando lleguemos a la simulación IoT (dispositivo, instante, variable, unidad y valor). Con eso quedamos en siete entidades, dentro del límite.



PR09 Monitoreo de huerto o ambiente - modelo de datos P02 (7 entidades). Las seis de la ficha C11 mas 'sensor' como dispositivo que mide una variable en una zona.

Figura 1. Modelo de datos del PR09 en P02. Las flechas indican la dirección de la llave foránea.

Hay dos rangos distintos a propósito. El rango físico vive en variable y sirve para rechazar valores imposibles (150 °C en temperatura ambiente): eso es una validación y responde 400. El umbral vive en umbral, es por sensor, y sirve para aceptar la lectura pero avisar: 38 °C es un valor posible, se guarda, y como pasa del máximo operativo de 32 °C se genera una alerta ALTA. Esa separación nos permitió mostrar en el mismo incremento una validación negativa y una regla de negocio con efecto persistido.

La columna origen de lectura solo admite 'manual' o 'simulado' (hay un CHECK en la base). Es nuestra respuesta al riesgo principal de la ficha: en la interfaz cada lectura lleva su etiqueta de procedencia y la unidad junto al valor, así que las dos filas de la semilla se ven claramente como simuladas y las que captura el observador como manuales. En anotación pasa algo parecido con autor_rol: la base solo acepta RESPONSABLE, OBSERVADOR o COORDINACION, que son los tres roles funcionales del proyecto.

Entidad	Para qué sirve	Restricciones que aplica PostgreSQL
zona	Cama, invernadero o parcela del huerto	nombre único; ubicación obligatoria
variable	Magnitud que se mide, con unidad y rango físico	clave única; valor_minimo < valor_maximo
sensor	Dispositivo que mide una variable en una zona	FK a zona y variable; código único
umbral	Rango operativo deseado de un sensor	uno por sensor; minimo < maximo
lectura	Medición con instante, valor y procedencia	FK a sensor; origen IN (manual, simulado); índice por sensor y fecha
alerta	Aviso generado por una lectura fuera de umbral	una por lectura; nivel IN (BAJA, ALTA)
anotacion	Nota de un rol sobre una zona, opcionalmente ligada a una lectura	FK a zona y lectura; rol IN (3 roles); texto de 3 a 300

5. Cómo está construida la aplicación

Seguimos la separación que se explica en M02: la JSP solo presenta, el Servlet coordina la petición, una clase de servicio aplica las reglas y abre la transacción, y los repositorios concentran el SQL. Así pudimos probar las reglas de validación con JUnit sin levantar Tomcat ni la base de datos, y ninguna vista tiene SQL ni credenciales.

Capa	Clases	Responsabilidad
Configuración	DbConfig, ApplicationContextListener	Lee DB_URL, DB_USER y DB_PASSWORD del entorno; libera el driver JDBC al parar
Modelo	Zona, Sensor, Lectura, Alerta, Anotacion	Objetos simples que viajan del repositorio a la JSP
Repositorio	SensorRepository, LecturaRepository, AnotacionRepository	PreparedStatement, try-with-resources, RETURNING id
Servicio	LecturaValidator, AnotacionValidator, LecturaService	Reglas de RF03 y RF06; transacción lectura + alerta
Web	LecturaServlet, AlertaServlet, AnotacionServlet, HealthServlet	GET/POST, códigos 200/303/400, forward a las vistas

Vistas	lecturas.jsp, alertas.jsp, anotaciones.jsp, error.jsp	JSTL con c:out para escapar HTML
--------	---	----------------------------------

Un ejemplo de cómo insertamos la lectura y la alerta dentro de la misma conexión, con autocommit apagado:

```
try (Connection cn = DbConfig.getConnection()) {
    cn.setAutoCommit(false);
    try {
        Sensor sensor = sensorRepository.findById(cn, sensorId).orElseThrow(...);
        LecturaValidator.validarRangoFisico(sensor, valor, errores); // 400 si falla
        long lecturaId = lecturaRepository.insertLectura(cn, sensor.getId(), valor, observacion);
        String nivel = LecturaValidator.nivelAlerta(sensor, valor); // BAJA, ALTA o null
        if (nivel != null) {
            lecturaRepository.insertAlerta(cn, lecturaId, nivel,
            LecturaValidator.mensajeAlerta(sensor, valor, nivel));
        }
        cn.commit();
    } catch (ValidacionException | SQLException | RuntimeException e) {
        cn.rollback();
        throw e;
    }
}
```

Y el INSERT correspondiente, siempre parametrizado:

```
String sql = "INSERT INTO lectura (sensor_id, valor, origen, observacion) VALUES (?, ?, 'manual', ?) RETURNING id";
try (PreparedStatement ps = cn.prepareStatement(sql)) {
    ps.setLong(1, sensorId);
    ps.setBigDecimal(2, valor);
    ps.setString(3, observacion);
    try (ResultSet rs = ps.executeQuery()) { rs.next(); return rs.getLong(1); }
}
```

Ruta	Método	Qué hace
/web1/	GET	Redirige a /lecturas
/web1/lecturas	GET	Catálogo de sensores con umbral e historial de lecturas
/web1/lecturas	POST	Registra una lectura; 303 si es válida, 400 con errores si no
/web1/alertas	GET	Alertas generadas
/web1/anotaciones	GET / POST	Zonas y anotaciones; alta con rol funcional
/web1/health	GET	JSON con estado de la conexión y versión de PostgreSQL; 503 si la base no responde

6. Instalación y despliegue

El paso a paso completo, con las dos opciones de base de datos (Docker o PostgreSQL ya instalado), está en el README del repositorio. Aquí va el resumen de lo que hicimos nosotros en Windows con Git Bash:

```
git clone https://github.com/zywoxxx/dsw-p02-monitoreo-huerto.git
cd dsw-p02-monitoreo-huerto
docker compose -f docker/docker-compose.yml up -d          # PostgreSQL 16 en localhost:5436, crea
tablas y semilla
cp scripts/setenv.bat.example "$CATALINA_HOME/bin/setenv.bat" # DB_URL, DB_USER, DB_PASSWORD
fuera del código
mvn clean package                                           # 14 pruebas, target/web1.war
cp target/web1.war "$CATALINA_HOME/webapps/" && "$CATALINA_HOME/bin/startup.bat"
curl http://localhost:8080/web1/health
```

Las credenciales de laboratorio (usuario huerto_app, contraseña huerto_dev) están solo en el docker-compose y en los archivos de ejemplo de setenv; el setenv real de Tomcat está en .gitignore. La aplicación las lee de las variables de entorno en DbConfig, de modo que el mismo WAR sirve en cualquier máquina.

Para no depender de que alguien repita todo a mano, hicimos scripts/verify-module.sh. Compila, corre las pruebas unitarias, revisa el contenido del WAR, consulta PostgreSQL, vuelve a desplegar en Tomcat esperando a que el contexto suba, y ejecuta con curl el recorrido completo (GET inicial, POST válido, tres POST inválidos, POST que genera alerta, GET con persistencia, alertas y anotaciones). Cada paso deja su salida en docs/evidencia/txt y se marca como VERIFICADO o NO_VERIFICADO. La última corrida terminó así:

```
verify-module.sh M02 - 2026-09-11T18:31:22-06:00
VERIFICADO | Java 11 disponible
VERIFICADO | mvn clean package genera target/web1.war con pruebas en verde
VERIFICADO | WAR contiene servlets, JSP, web.xml y driver JDBC
VERIFICADO | PostgreSQL responde y el modelo de 7 tablas existe
VERIFICADO | web1 desplegado y respondiendo en http://localhost:8080/web1
VERIFICADO | /health responde 200 y db=UP
VERIFICADO | GET /lecturas inicial responde 200 con catalogo y lecturas
VERIFICADO | POST valido responde 303 y redirige a la lectura creada
VERIFICADO | POST invalido (valor abc) responde 400 con mensaje de validacion
VERIFICADO | POST invalido (vacio) responde 400 con dos errores
VERIFICADO | POST invalido (150 C) responde 400 por rango fisico
VERIFICADO | POST 38 C responde 303 y genera alerta ALTA
VERIFICADO | GET posterior muestra las lecturas persistidas (2 -> 4 filas)
VERIFICADO | GET /alertas lista la alerta generada
VERIFICADO | GET /anotaciones responde 200 con zonas y anotaciones
VERIFICADO | POST anotacion valida responde 303
VERIFICADO | POST anotacion invalida (rol ADMIN, texto corto) responde 400
VERIFICADO | Las lecturas y la anotacion del protocolo existen en PostgreSQL
Fallos: 0
```

7. Pruebas y evidencia

7.1 Pruebas unitarias

Las reglas de validación están en clases sin dependencias de Tomcat ni de JDBC, así que se prueban con JUnit 5 durante mvn package. Son 14 casos: 10 para lecturas (valor válido en rango, alerta ALTA, alerta BAJA, valor vacío, no numérico, tres decimales, fuera del rango físico, sensor inválido, sensor inactivo y

observación demasiado larga) y 4 para anotaciones (caso válido, rol fuera de los tres permitidos, texto vacío o muy largo, zona y lectura inválidas).

```
-----
Test set: mx.uv.dsw.huerto.service.AnotacionValidatorTest
-----
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.053 s -- in
mx.uv.dsw.huerto.service.AnotacionValidatorTest
-----
Test set: mx.uv.dsw.huerto.service.LecturaValidatorTest
-----
Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.016 s -- in
mx.uv.dsw.huerto.service.LecturaValidatorTest
-----
```

7.2 Recorrido del flujo principal

Las capturas se tomaron con un navegador real contra la aplicación desplegada, después de dejar la base con la semilla limpia. Primero la pantalla inicial: cinco sensores en dos zonas y las dos lecturas simuladas de la semilla, cada una con su etiqueta de procedencia.

PR09 · Monitoreo de huerto o ambiente
Incremento P02 · JSP / Servlet 4.0 · Tomcat 9 · PostgreSQL 16

Lecturas
Alertas
Zonas y anotaciones
Salud (JSON)

Registrar lectura manual
RF03: selecciona un sensor y captura el valor medido. El sistema valida el rango físico de la variable, etiqueta la lectura como **manual** y genera una alerta si el valor sale del umbral operativo (RF04, RF06).

Sensor *
-- Selecciona un sensor --
Valor medido *
Ej. 24.5

Observacion (opcional, max. 200)

Registrar lectura

Sensores por zona y variable (RF01, RF02, RF04)

Codigo	Variable	Unidad	Zona	Rango físico	Umbral operativo
SEN-A-HSUE-01	Humedad de suelo	%	Cama A	[0.00, 100.00]	[40.00, 80.00]
SEN-A-TEMP-01	Temperatura ambiente	C	Cama A	[-10.00, 60.00]	[15.00, 32.00]
SEN-I1-HAIR-01	Humedad relativa	%	Invernadero 1	[0.00, 100.00]	[50.00, 85.00]
SEN-I1-LUZ-01	Luminosidad	lux	Invernadero 1	[0.00, 100000.00]	[2000.00, 60000.00]
SEN-I1-TEMP-01	Temperatura ambiente	C	Invernadero 1	[-10.00, 60.00]	[18.00, 30.00]

Historial de lecturas (RF05)

#	Sensor	Variable	Zona	Valor	Procedencia	Observacion	Registrada	Alerta
2	SEN-A-HSUE-01	Humedad de suelo	Cama A	55.00 %	simulado	Carga inicial del laboratorio	2026-09-12 00:24:51	en rango
1	SEN-A-TEMP-01	Temperatura ambiente	Cama A	24.50 C	simulado	Carga inicial del laboratorio	2026-09-12 00:24:51	en rango

Desarrollo de Sistemas Web · DSW-19559 · Datos ficticios de laboratorio.

Figura 2. GET /web1/lecturas con la semilla inicial (RF01, RF02, RF04, RF05).

Después llenamos el formulario con el sensor de temperatura de la Cama A y 27.5 °C, un valor dentro del umbral.

PR09 · Monitoreo de huerto o ambiente
Incremento P02 · JSP / Servlet 4.0 · Tomcat 9 · PostgreSQL 16

Lecturas
Alertas
Zonas y anotaciones
Salud (JSON)

Registrar lectura manual

RF03: selecciona un sensor y captura el valor medido. El sistema valida el rango físico de la variable, etiqueta la lectura como **manual** y genera una alerta si el valor sale del umbral operativo (RF04, RF06).

Sensor *

SEN-A-TEMP-01 - Temperatura ambiente (C) - Cama A

▼

Valor medido *

27.5

Observacion (opcional, max. 200)

Lectura manual de prueba (captura)

Registrar lectura

Sensores por zona y variable (RF01, RF02, RF04)

Codigo	Variable	Unidad	Zona	Rango físico	Umbral operativo
SEN-A-HSUE-01	Humedad de suelo	%	Cama A	[0.00, 100.00]	[40.00, 80.00]
SEN-A-TEMP-01	Temperatura ambiente	C	Cama A	[-10.00, 60.00]	[15.00, 32.00]
SEN-I1-HAIR-01	Humedad relativa	%	Invernadero 1	[0.00, 100.00]	[50.00, 85.00]
SEN-I1-LUZ-01	Luminosidad	lux	Invernadero 1	[0.00, 100000.00]	[2000.00, 60000.00]
SEN-I1-TEMP-01	Temperatura ambiente	C	Invernadero 1	[-10.00, 60.00]	[18.00, 30.00]

Historial de lecturas (RF05)

#	Sensor	Variable	Zona	Valor	Procedencia	Observacion	Registrada	Alerta
2	SEN-A-HSUE-01	Humedad de suelo	Cama A	55.00 %	simulado	Carga inicial del laboratorio	2026-09-12 00:24:51	en rango
1	SEN-A-TEMP-01	Temperatura ambiente	Cama A	24.50 C	simulado	Carga inicial del laboratorio	2026-09-12 00:24:51	en rango

Desarrollo de Sistemas Web · DSW-19559 · Datos ficticios de laboratorio.

Figura 3. Formulario antes de enviar (RF03).

PR09 · Monitoreo de huerto o ambiente

Incremento P02 · JSP / Servlet 4.0 · Tomcat 9 · PostgreSQL 16

Lecturas

Alertas

Zonas y anotaciones

Salud (JSON)

Lectura #5 registrada correctamente.

Registrar lectura manual

RF03: selecciona un sensor y captura el valor medido. El sistema valida el rango físico de la variable, etiqueta la lectura como **manual** y genera una alerta si el valor sale del umbral operativo (RF04, RF06).

Sensor *

-- Selecciona un sensor --

Valor medido *

Ej. 24.5

Observacion (opcional, max. 200)

Registrar lectura

Sensores por zona y variable (RF01, RF02, RF04)

Codigo	Variable	Unidad	Zona	Rango físico	Umbral operativo
SEN-A-HSUE-01	Humedad de suelo	%	Cama A	[0.00, 100.00]	[40.00, 80.00]
SEN-A-TEMP-01	Temperatura ambiente	C	Cama A	[-10.00, 60.00]	[15.00, 32.00]
SEN-I1-HAIR-01	Humedad relativa	%	Invernadero 1	[0.00, 100.00]	[50.00, 85.00]
SEN-I1-LUZ-01	Luminosidad	lux	Invernadero 1	[0.00, 100000.00]	[2000.00, 60000.00]
SEN-I1-TEMP-01	Temperatura ambiente	C	Invernadero 1	[-10.00, 60.00]	[18.00, 30.00]

Historial de lecturas (RF05)

#	Sensor	Variable	Zona	Valor	Procedencia	Observacion	Registrada	Alerta
5	SEN-A-TEMP-01	Temperatura ambiente	Cama A	27.50 C	manual	Lectura manual de prueba (captura)	2026-09-12 00:31:24	en rango
2	SEN-A-HSUE-01	Humedad de suelo	Cama A	55.00 %	simulado	Carga inicial del laboratorio	2026-09-12 00:24:51	en rango
1	SEN-A-TEMP-01	Temperatura ambiente	Cama A	24.50 C	simulado	Carga inicial del laboratorio	2026-09-12 00:24:51	en rango

Desarrollo de Sistemas Web · DSW-19559 · Datos ficticios de laboratorio.

Figura 4. Resultado del POST válido: mensaje de confirmación y la lectura nueva marcada como manual (CA01).

Con 38 °C la lectura también es válida (está dentro del rango físico) pero supera el umbral de 32 °C, así que además del mensaje de éxito aparece el aviso de alerta y la fila queda resaltada.

PR09 · Monitoreo de huerto o ambiente

Incremento P02 · JSP / Servlet 4.0 · Tomcat 9 · PostgreSQL 16

[Lecturas](#)
[Alertas](#)
[Zonas y anotaciones](#)
[Salud \(JSON\)](#)

Lectura #6 registrada correctamente.

Se genero una alerta de nivel ALTA porque el valor esta fuera del umbral del sensor.

Registrar lectura manual

RF03: selecciona un sensor y captura el valor medido. El sistema valida el rango fisico de la variable, etiqueta la lectura como **manual** y genera una alerta si el valor sale del umbral operativo (RF04, RF06).

Sensor *

-- Selecciona un sensor --

Valor medido *

Ej. 24.5

Observacion (opcional, max. 200)

Registrar lectura

Sensores por zona y variable (RF01, RF02, RF04)

Codigo	Variable	Unidad	Zona	Rango fisico	Umbral operativo
SEN-A-HSUE-01	Humedad de suelo	%	Cama A	[0.00, 100.00]	[40.00, 80.00]
SEN-A-TEMP-01	Temperatura ambiente	C	Cama A	[-10.00, 60.00]	[15.00, 32.00]
SEN-I1-HAIR-01	Humedad relativa	%	Invernadero 1	[0.00, 100.00]	[50.00, 85.00]
SEN-I1-LUZ-01	Luminosidad	lux	Invernadero 1	[0.00, 100000.00]	[2000.00, 60000.00]
SEN-I1-TEMP-01	Temperatura ambiente	C	Invernadero 1	[-10.00, 60.00]	[18.00, 30.00]

Historial de lecturas (RF05)

#	Sensor	Variable	Zona	Valor	Procedencia	Observacion	Registrada	Alerta
6	SEN-A-TEMP-01	Temperatura ambiente	Cama A	38.00 C	manual	Tarde calurosa (captura)	2026-09-12 00:31:25	ALTA
5	SEN-A-TEMP-01	Temperatura ambiente	Cama A	27.50 C	manual	Lectura manual de prueba (captura)	2026-09-12 00:31:24	en rango
2	SEN-A-HSUE-01	Humedad de suelo	Cama A	55.00 %	simulado	Carga inicial del laboratorio	2026-09-12 00:24:51	en rango
1	SEN-A-TEMP-01	Temperatura ambiente	Cama A	24.50 C	simulado	Carga inicial del laboratorio	2026-09-12 00:24:51	en rango

Desarrollo de Sistemas Web · DSW-19559 · Datos ficticios de laboratorio.

Figura 5. POST con 38 °C: lectura guardada y alerta ALTA generada en la misma transacción (RF06).

PR09 · Monitoreo de huerto o ambiente

Incremento P02 · JSP / Servlet 4.0 · Tomcat 9 · PostgreSQL 16

[Lecturas](#)
[Alertas](#)
[Zonas y anotaciones](#)
[Salud \(JSON\)](#)

Alertas generadas (RF06)

Cada alerta nace de una lectura cuyo valor quedo fuera del umbral operativo del sensor (RF04).

#	Lectura	Sensor	Zona	Valor	Nivel	Mensaje	Atendida	Creada
2	#6	SEN-A-TEMP-01	Cama A	38.00 C	ALTA	Temperatura ambiente en Cama A: 38 C por encima del maximo 32.00 C.	No	2026-09-12 00:31:25

Desarrollo de Sistemas Web · DSW-19559 · Datos ficticios de laboratorio.

Figura 6. Vista /web1/alertas con la alerta recién creada.

En la consola, la respuesta del servidor al POST válido fue la siguiente (el 303 y la cabecera Location son lo que demuestra el patrón POST-Redirect-GET):

```
HTTP/1.1 303
Location: /web1/lecturas?creada=3
Content-Length: 0
Date: Sat, 12 Sep 2026 00:31:20 GMT
```

7.3 Validaciones negativas

Probamos los casos inválidos que marca la ficha y algunos más. En todos el servidor responde 400, muestra el motivo y conserva lo que ya se había escrito; en la base no cambia nada.

PR09 · Monitoreo de huerto o ambiente
Incremento P02 · JSP / Servlet 4.0 · Tomcat 9 · PostgreSQL 16

Lecturas
Alertas
Zonas y anotaciones
Salud (JSON)

La lectura no se registro. Corrige lo siguiente:

- El valor debe ser numerico (ejemplo: 24.5).

Registrar lectura manual
RF03: selecciona un sensor y captura el valor medido. El sistema valida el rango fisico de la variable, etiqueta la lectura como **manual** y genera una alerta si el valor sale del umbral operativo (RF04, RF06).

Sensor *
Valor medido *

SEN-A-TEMP-01 - Temperatura ambiente (C) - Cama A
abc

Observacion (opcional, max. 200)

Registrar lectura

Sensores por zona y variable (RF01, RF02, RF04)

Codigo	Variable	Unidad	Zona	Rango fisico	Umbral operativo
SEN-A-HSUE-01	Humedad de suelo	%	Cama A	[0.00, 100.00]	[40.00, 80.00]
SEN-A-TEMP-01	Temperatura ambiente	C	Cama A	[-10.00, 60.00]	[15.00, 32.00]
SEN-I1-HAIR-01	Humedad relativa	%	Invernadero 1	[0.00, 100.00]	[50.00, 85.00]
SEN-I1-LUZ-01	Luminosidad	lux	Invernadero 1	[0.00, 100000.00]	[2000.00, 60000.00]
SEN-I1-TEMP-01	Temperatura ambiente	C	Invernadero 1	[-10.00, 60.00]	[18.00, 30.00]

Historial de lecturas (RF05)

#	Sensor	Variable	Zona	Valor	Procedencia	Observacion	Registrada	Alerta
5	SEN-A-TEMP-01	Temperatura ambiente	Cama A	27.50 C	manual	Lectura manual de prueba (captura)	2026-09-12 00:31:24	en rango
2	SEN-A-HSUE-01	Humedad de suelo	Cama A	55.00 %	simulado	Carga inicial del laboratorio	2026-09-12 00:24:51	en rango
1	SEN-A-TEMP-01	Temperatura ambiente	Cama A	24.50 C	simulado	Carga inicial del laboratorio	2026-09-12 00:24:51	en rango

Desarrollo de Sistemas Web · DSW-19559 · Datos ficticios de laboratorio.

Figura 7. Valor no numérico (abc): 400 con mensaje (CA02).

PR09 · Monitoreo de huerto o ambiente

Incremento P02 · JSP / Servlet 4.0 · Tomcat 9 · PostgreSQL 16

Lecturas

Alertas

Zonas y anotaciones

Salud (JSON)

La lectura no se registro. Corrige lo siguiente:

- El valor 150 C esta fuera del rango fisico permitido para Temperatura ambiente [-10.00, 60.00].

Registrar lectura manual

RF03: selecciona un sensor y captura el valor medido. El sistema valida el rango fisico de la variable, etiqueta la lectura como **manual** y genera una alerta si el valor sale del umbral operativo (RF04, RF06).

Sensor *

SEN-A-TEMP-01 - Temperatura ambiente (C) - Cama A

Valor medido *

150

Observacion (opcional, max. 200)

Registrar lectura

Sensores por zona y variable (RF01, RF02, RF04)

Codigo	Variable	Unidad	Zona	Rango fisico	Umbral operativo
SEN-A-HSUE-01	Humedad de suelo	%	Cama A	[0.00, 100.00]	[40.00, 80.00]
SEN-A-TEMP-01	Temperatura ambiente	C	Cama A	[-10.00, 60.00]	[15.00, 32.00]
SEN-I1-HAIR-01	Humedad relativa	%	Invernadero 1	[0.00, 100.00]	[50.00, 85.00]
SEN-I1-LUZ-01	Luminosidad	lux	Invernadero 1	[0.00, 100000.00]	[2000.00, 60000.00]
SEN-I1-TEMP-01	Temperatura ambiente	C	Invernadero 1	[-10.00, 60.00]	[18.00, 30.00]

Historial de lecturas (RF05)

#	Sensor	Variable	Zona	Valor	Procedencia	Observacion	Registrada	Alerta
5	SEN-A-TEMP-01	Temperatura ambiente	Cama A	27.50 C	manual	Lectura manual de prueba (captura)	2026-09-12 00:31:24	en rango
2	SEN-A-HSUE-01	Humedad de suelo	Cama A	55.00 %	simulado	Carga inicial del laboratorio	2026-09-12 00:24:51	en rango
1	SEN-A-TEMP-01	Temperatura ambiente	Cama A	24.50 C	simulado	Carga inicial del laboratorio	2026-09-12 00:24:51	en rango

Desarrollo de Sistemas Web · DSW-19559 · Datos ficticios de laboratorio.

Figura 8. 150 °C está fuera del rango físico de la variable: se rechaza (prueba mínima de la ficha).

PR09 · Monitoreo de huerto o ambiente

Incremento P02 · JSP / Servlet 4.0 · Tomcat 9 · PostgreSQL 16

[Lecturas](#)
[Alertas](#)
[Zonas y anotaciones](#)
[Salud \(JSON\)](#)
La lectura no se registro. Corrige lo siguiente:

- Debe seleccionar un sensor.
- El valor de la lectura es obligatorio.

Registrar lectura manual

RF03: selecciona un sensor y captura el valor medido. El sistema valida el rango físico de la variable, etiqueta la lectura como **manual** y genera una alerta si el valor sale del umbral operativo (RF04, RF06).

Sensor *

-- Selecciona un sensor --

Valor medido *

Ej. 24.5

Observacion (opcional, max. 200)

Registrar lectura

Sensores por zona y variable (RF01, RF02, RF04)

Codigo	Variable	Unidad	Zona	Rango fisico	Umbral operativo
SEN-A-HSUE-01	Humedad de suelo	%	Cama A	[0.00, 100.00]	[40.00, 80.00]
SEN-A-TEMP-01	Temperatura ambiente	C	Cama A	[-10.00, 60.00]	[15.00, 32.00]
SEN-I1-HAIR-01	Humedad relativa	%	Invernadero 1	[0.00, 100.00]	[50.00, 85.00]
SEN-I1-LUZ-01	Luminosidad	lux	Invernadero 1	[0.00, 100000.00]	[2000.00, 60000.00]
SEN-I1-TEMP-01	Temperatura ambiente	C	Invernadero 1	[-10.00, 60.00]	[18.00, 30.00]

Historial de lecturas (RF05)

#	Sensor	Variable	Zona	Valor	Procedencia	Observacion	Registrada	Alerta
5	SEN-A-TEMP-01	Temperatura ambiente	Cama A	27.50 C	manual	Lectura manual de prueba (captura)	2026-09-12 00:31:24	en rango
2	SEN-A-HSUE-01	Humedad de suelo	Cama A	55.00 %	simulado	Carga inicial del laboratorio	2026-09-12 00:24:51	en rango
1	SEN-A-TEMP-01	Temperatura ambiente	Cama A	24.50 C	simulado	Carga inicial del laboratorio	2026-09-12 00:24:51	en rango

Desarrollo de Sistemas Web · DSW-19559 · Datos ficticios de laboratorio.

Figura 9. Campos vacíos: dos mensajes de error a la vez.

Por curl también comprobamos un sensor inexistente (id 999), que responde 400 con 'El sensor seleccionado no existe'. Este es el encabezado de la respuesta al valor no numérico:

```
HTTP/1.1 400
Set-Cookie: JSESSIONID=36B0543EF2A1AA11F3AF617058286CD8; Path=/web1; HttpOnly
Content-Type: text/html; charset=UTF-8
Content-Length: 7843
```

7.4 Zonas y anotaciones

La página de anotaciones lista las zonas del huerto y permite que cualquiera de los tres roles deje una nota sobre una zona, opcionalmente ligada al número de una lectura. El rol se valida contra la lista de tres y la base lo vuelve a comprobar con un CHECK.

PR09 · Monitoreo de huerto o ambiente
Incremento P02 · JSP / Servlet 4.0 · Tomcat 9 · PostgreSQL 16

LecturasAlertasZonas y anotacionesSalud (JSON)

Zonas del huerto (RF01)

#	Zona	Cultivo	Ubicacion
1	Cama A	Jitomate	Huerto escolar demo - patio norte (ficticio)
2	Invernadero 1	Lechuga	Huerto escolar demo - invernadero (ficticio)

Registrar anotacion (RF06)

Los tres roles funcionales del PR09 pueden anotar observaciones sobre una zona y, si aplica, asociarlas al numero de una lectura.

Zona *
-- Selecciona una zona --

Rol que anota *
-- Selecciona un rol --

Lectura relacionada (opcional, numero)

Anotacion * (3 a 300 caracteres)

Registrar anotacion

Anotaciones recientes

#	Zona	Rol	Anotacion	Lectura	Creada
1	Cama A	RESPONSABLE	Sensores instalados y calibrados con datos ficticios de laboratorio.		2026-09-12 00:24:51

Desarrollo de Sistemas Web · DSW-19559 · Datos ficticios de laboratorio.

Figura 10. GET /web1/anotaciones con las zonas y la anotación de la semilla (RF01, RF06).

PR09 · Monitoreo de huerto o ambiente
 Incremento P02 · JSP / Servlet 4.0 · Tomcat 9 · PostgreSQL 16

Lecturas
 Alertas
 Zonas y anotaciones
 Salud (JSON)

Anotacion #3 registrada correctamente.

Zonas del huerto (RF01)

#	Zona	Cultivo	Ubicacion
1	Cama A	Jitomate	Huerto escolar demo - patio norte (ficticio)
2	Invernadero 1	Lechuga	Huerto escolar demo - invernadero (ficticio)

Registrar anotacion (RF06)
 Los tres roles funcionales del PR09 pueden anotar observaciones sobre una zona y, si aplica, asociarlas al numero de una lectura.

Zona *
 -- Selecciona una zona --

Rol que anota *
 -- Selecciona un rol --

Lectura relacionada (opcional, numero)

Anotacion * (3 a 300 caracteres)

Registrar anotacion

Anotaciones recientes

#	Zona	Rol	Anotacion	Lectura	Creada
3	Cama A	OBSERVADOR	[prueba] Hojas con manchas en la cama A (captura)		2026-09-12 00:31:25
1	Cama A	RESPONSABLE	Sensores instalados y calibrados con datos ficticios de laboratorio.		2026-09-12 00:24:51

Desarrollo de Sistemas Web · DSW-19559 · Datos ficticios de laboratorio.

Figura 11. Anotación registrada por el rol OBSERVADOR.

PR09 · Monitoreo de huerto o ambiente
Incremento P02 · JSP / Servlet 4.0 · Tomcat 9 · PostgreSQL 16

Lecturas
Alertas
Zonas y anotaciones
Salud (JSON)

La anotacion no se registro. Corrige lo siguiente:

- La anotacion debe tener al menos 3 caracteres.

Zonas del huerto (RF01)

#	Zona	Cultivo	Ubicacion
1	Cama A	Jitomate	Huerto escolar demo - patio norte (ficticio)
2	Invernadero 1	Lechuga	Huerto escolar demo - invernadero (ficticio)

Registrar anotacion (RF06)
Los tres roles funcionales del PR09 pueden anotar observaciones sobre una zona y, si aplica, asociarlas al numero de una lectura.

Zona *
Cama A - Jitomate

Rol que anota *
COORDINACION

Lectura relacionada (opcional, numero)

Anotacion * (3 a 300 caracteres)
ok

Registrar anotacion

Anotaciones recientes

#	Zona	Rol	Anotacion	Lectura	Creada
3	Cama A	OBSERVADOR	[prueba] Hojas con manchas en la cama A (captura)		2026-09-12 00:31:25
1	Cama A	RESPONSABLE	Sensores instalados y calibrados con datos ficticios de laboratorio.		2026-09-12 00:24:51

Desarrollo de Sistemas Web · DSW-19559 · Datos ficticios de laboratorio.

Figura 12. Texto de dos caracteres: 400 con el mensaje de longitud mínima.

7.5 Despliegue en Tomcat y persistencia en PostgreSQL

La ruta de salud nos sirvió durante todo el desarrollo para saber si Tomcat estaba conectado a la base. Devuelve la versión del servidor PostgreSQL y el total de lecturas; si la base no responde, devuelve 503 con el motivo.

Así se ve la respuesta con curl (la captura del navegador está en docs/evidencia/img/09_health_json.png, pero el JSON se lee mejor en texto):

```
HTTP/1.1 200
Content-Type: application/json;charset=UTF-8
Content-Length: 259
Date: Sat, 12 Sep 2026 00:31:19 GMT

{"status":"UP","app":"web1 PR09 monitoreo de
huerto","db":"UP","dbUrl":"jdbc:postgresql://localhost:5436/huerto_db","postgres":"PostgreSQL
16.15 (Debian 16.15-1.pgdg13+2) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0,
64-bit","lecturas":2}
```

Del lado de la base de datos, estas son capturas de sesiones psql contra el PostgreSQL 16 del contenedor (puerto 5436) tomadas después de las capturas de la aplicación. En la primera se ve la conexión, las siete tablas con su dueño y los conteos; en la segunda, la estructura de lectura y anotación con sus llaves, restricciones CHECK e índices; en la tercera, las lecturas con su procedencia, la alerta y las anotaciones, que coinciden con las figuras anteriores.

```

psql - PostgreSQL 16 (D...
Está conectado a la base de datos «huerto_db» como el usuario «huerto_app» en el servidor «127.0.0.1» port «5436».
version
PostgreSQL 16.15 (Debian 16.15-1.pgdgl3+2) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
(1 fila)

Listado de relaciones
Esquema | Nombre | Tipo | Dueño
-----+-----+-----+-----
public | alerta | tabla | huerto_app
public | anotacion | tabla | huerto_app
public | lectura | tabla | huerto_app
public | sensor | tabla | huerto_app
public | umbral | tabla | huerto_app
public | variable | tabla | huerto_app
public | zona | tabla | huerto_app
(7 filas)

entidad | registros
-----+-----
zona | 2
variable | 4
sensor | 5
umbral | 5
lectura | 4
alerta | 1
anotacion | 2
(7 filas)

C:\Users\alexg>

```

Figura 13. psql en PostgreSQL 16 (Docker): conexión, tablas y conteos por entidad.

```

psql - PostgreSQL 16 (D...
+

Tabla «public.lectura»
+-----+-----+-----+-----+-----+
| Columna | Tipo | Ordenamiento | Nullable | Por omisión |
+-----+-----+-----+-----+-----+
| id       | bigint | | not null | generated always as identity |
| sensor_id | bigint | | not null | |
| valor    | numeric(10,2) | | not null | |
| origen   | character varying(10) | | not null | 'manual'::character varying |
| observacion | character varying(200) | | not null | |
| registrado_en | timestamp with time zone | | not null | now() |
+-----+-----+-----+-----+-----+
Índices:
"lectura_pkey" PRIMARY KEY, btree (id)
"idx_lectura_sensor_fecha" btree (sensor_id, registrado_en DESC)
Restricciones CHECK:
"ck_lectura_origen" CHECK (origen::text = ANY (ARRAY['manual'::character varying, 'simulado'::character varying]::text[]))
Restricciones de llave foránea:
"lectura_sensor_id_fkey" FOREIGN KEY (sensor_id) REFERENCES sensor(id) ON DELETE CASCADE
Referenciada por:
TABLE "alerta" CONSTRAINT "alerta_lectura_id_fkey" FOREIGN KEY (lectura_id) REFERENCES lectura(id) ON DELETE CASCADE
TABLE "anotacion" CONSTRAINT "anotacion_lectura_id_fkey" FOREIGN KEY (lectura_id) REFERENCES lectura(id) ON DELETE SET NULL

Tabla «public.anotacion»
+-----+-----+-----+-----+-----+
| Columna | Tipo | Ordenamiento | Nullable | Por omisión |
+-----+-----+-----+-----+-----+
| id       | bigint | | not null | generated always as identity |
| zona_id  | bigint | | not null | |
| lectura_id | bigint | | not null | |
| autor_rol | character varying(15) | | not null | |
| texto    | character varying(300) | | not null | |
| creada_en | timestamp with time zone | | not null | now() |
+-----+-----+-----+-----+-----+
Índices:
"anotacion_pkey" PRIMARY KEY, btree (id)
"idx_anotacion_zona_fecha" btree (zona_id, creada_en DESC)
Restricciones CHECK:
"ck_anotacion_rol" CHECK (autor_rol::text = ANY (ARRAY['RESPONSABLE'::character varying, 'OBSERVADOR'::character varying, 'COORDINACION'::character varying]::text[]))
"ck_anotacion_texto" CHECK (length(trim(both from texto)) >= 3)
Restricciones de llave foránea:
"anotacion_lectura_id_fkey" FOREIGN KEY (lectura_id) REFERENCES lectura(id) ON DELETE SET NULL
"anotacion_zona_id_fkey" FOREIGN KEY (zona_id) REFERENCES zona(id) ON DELETE CASCADE

C:\Users\alexg>

```

Figura 14. psql: estructura de las tablas lectura y anotación (llaves foráneas, CHECK e índices).

The screenshot shows a PostgreSQL terminal window with the following tables and data:

id	codigo	variable	zona	valor	unidad	origen	observacion	registrado_en	alerta
6	SEN-A-TEMP-01	Temperatura ambiente	Cama A	38.00	C	manual	Tarde calurosa (captura)	2026-09-12 00:31:25.370416+00	ALTA
5	SEN-A-TEMP-01	Temperatura ambiente	Cama A	27.50	C	manual	Lectura manual de prueba (captura)	2026-09-12 00:31:24.491533+00	
1	SEN-A-TEMP-01	Temperatura ambiente	Cama A	24.50	C	simulado	Carga inicial del laboratorio	2026-09-12 00:24:51.395115+00	
2	SEN-A-HSUE-01	Humedad de suelo	Cama A	55.00	%	simulado	Carga inicial del laboratorio	2026-09-12 00:24:51.395115+00	

(4 filas)

id	codigo	nivel	mensaje	atendida	creada_en
2	SEN-A-TEMP-01	ALTA	Temperatura ambiente en Cama A: 38 C por encima del maximo 32.00 C.	f	2026-09-12 00:31:25.370416+00

(1 fila)

id	zona	autor_rol	texto	lectura_id	creada_en
3	Cama A	OBSERVADOR	[prueba] Hojas con manchas en la cama A (captura)		2026-09-12 00:31:25.979371+00
1	Cama A	RESPONSABLE	Sensores instalados y calibrados con datos ficticios de laboratorio.		2026-09-12 00:24:51.395115+00

(2 filas)

C:\Users\alexg>

Figura 15. psql: lecturas con procedencia y alerta, alertas y anotaciones registradas.

Estas son las líneas del log de Tomcat en las que se ve el despliegue del WAR (las rutas locales se sustituyeron por marcadores para no incluir rutas privadas):

```
11-Sep-2026 17:39:45.107 INFO [main] org.apache.catalina.startup.Catalina.start Server startup in
[1785] milliseconds
11-Sep-2026 17:42:55.008 INFO [Catalina-utility-2]
org.apache.catalina.startup.HostConfig.deployWAR Despliegue del archivo
[<CATALINA_HOME>\webapps\web1.war] de la aplicación web
11-Sep-2026 17:44:05.345 INFO [Catalina-utility-1]
org.apache.catalina.startup.HostConfig.deployWAR Despliegue del archivo
[<CATALINA_HOME>\webapps\web1.war] de la aplicación web
11-Sep-2026 17:45:15.699 INFO [Catalina-utility-1]
org.apache.catalina.startup.HostConfig.deployWAR Despliegue del archivo
[<CATALINA_HOME>\webapps\web1.war] de la aplicación web
11-Sep-2026 17:54:16.551 INFO [Catalina-utility-1]
org.apache.catalina.startup.HostConfig.deployWAR Despliegue del archivo
[<CATALINA_HOME>\webapps\web1.war] de la aplicación web
11-Sep-2026 18:31:14.416 INFO [Catalina-utility-2]
org.apache.catalina.startup.HostConfig.deployWAR Despliegue del archivo
[<CATALINA_HOME>\webapps\web1.war] de la aplicación web
```

También probamos la Opción B del README con el PostgreSQL 17.11 que uno de nosotros tiene instalado en Windows (puerto 5433): creamos el rol y la base con psql, corrimos los mismos scripts de esquema y semilla, cambiamos solo DB_URL en el setenv de Tomcat y la misma aplicación, sin recompilar, quedó conectada. Estas son las líneas clave de esa prueba (el archivo completo es docs/evidencia/txt/19_postgres_nativo_opcionB.txt):

```
# Opcion B del README: PostgreSQL 17 instalado en Windows (servicio postgresql-x64-17, puerto 5433)
# Fecha: 2026-09-11T19:33:20-06:00. Comandos ejecutados (sin contraseñas):
# psql -U postgres -p 5433 -c "CREATE ROLE huerto_app LOGIN PASSWORD '...'"
# psql -U postgres -p 5433 -c "CREATE DATABASE huerto_db OWNER huerto_app"
# psql -U huerto_app -p 5433 -d huerto_db -f sql/01_schema.sql
# psql -U huerto_app -p 5433 -d huerto_db -f sql/02_seed.sql
# psql -U huerto_app -p 5433 -d huerto_db -f sql/03_consultas_verificacion.sql
PostgreSQL 17.11 on x86_64-windows, compiled by msvc-19.44.35228, 64-bit
id | codigo | variable | zona | valor | unidad | origen | observacion
|-----|-----|-----|-----|-----|-----|-----|-----
| registrado_en | alerta |
|-----|-----|-----|-----|-----|-----|-----|-----
1 | SEN-A-TEMP-01 | Temperatura ambiente | Cama A | 24.50 | C | simulado | Carga inicial del laboratorio | 2026-09-11 19:33:20.102458-06 |
2 | SEN-A-HSUE-01 | Humedad de suelo | Cama A | 55.00 | % | simulado | Carga inicial del laboratorio | 2026-09-11 19:33:20.102458-06 |
id | codigo | nivel | mensaje | atendida | creada_en
|-----|-----|-----|-----|-----|-----|
id | zona | autor_rol | texto
lectura_id | creada_en
|-----|-----|-----|-----|
1 | Cama A | RESPONSABLE | Sensores instalados y calibrados con datos ficticios de laboratorio. |
| 2026-09-11 19:33:20.102458-06
# --- La aplicacion web1 conectada al PostgreSQL 17 nativo
(DB_URL=jdbc:postgresql://localhost:5433/huerto_db) ---
$ curl -s http://localhost:8080/web1/health
{"status":"UP","app":"web1 PR09 monitoreo de huerto","db":"UP","dbUrl":"jdbc:postgresql://localhost:5433/huerto_db","postgres":"PostgreSQL 17.11 on x86_64-windows, compiled by msvc-19.44.35228, 64-bit","lecturas":2}
$ curl -i -X POST -d "sensorId=2&valor=29.5&observacion=Opcion+B+PostgreSQL+17+nativo" http://localhost:8080/web1/lecturas
HTTP/1.1 303
Location: /web1/lecturas?creada=3
$ curl -i -X POST -d "sensorId=2&valor=abc" http://localhost:8080/web1/lecturas (negativa)
HTTP/1.1 400
<li>El valor debe ser numerico (ejemplo: 24.5).</li>
$ psql -U huerto_app -p 5433 -d huerto_db -c "SELECT id, valor, origen, observacion, registrado_en FROM lectura ORDER BY id DESC LIMIT 3"
id | valor | origen | observacion | registrado_en
|-----|-----|-----|-----|-----|
3 | 29.50 | manual | Opcion B PostgreSQL 17 nativo | 2026-09-11 19:42:13.42885-06
2 | 55.00 | simulado | Carga inicial del laboratorio | 2026-09-11 19:33:20.102458-06
1 | 24.50 | simulado | Carga inicial del laboratorio | 2026-09-11 19:33:20.102458-06
(3 filas)
```

```

psql - PostgreSQL 17 ins x + v
Está conectado a la base de datos «huerto_db» como el usuario «huerto_app» en el servidor «127.0.0.1» port «5433».
version
-----
PostgreSQL 17.11 on x86_64-windows, compiled by msvc-19.44.35228, 64-bit
(1 fila)

Listado de relaciones
Esquema | Nombre | Tipo | Dueño
-----+-----+-----+-----
public | alerta | tabla | huerto_app
public | anotacion | tabla | huerto_app
public | lectura | tabla | huerto_app
public | sensor | tabla | huerto_app
public | umbral | tabla | huerto_app
public | variable | tabla | huerto_app
public | zona | tabla | huerto_app
(7 filas)

id | valor | origen | observacion | registrado_en
---+---+---+---+---
3 | 29.50 | manual | Opcion B PostgreSQL 17 nativo | 2026-09-11 19:42:13.42885-06
2 | 55.00 | simulado | Carga inicial del laboratorio | 2026-09-11 19:33:20.102458-06
1 | 24.50 | simulado | Carga inicial del laboratorio | 2026-09-11 19:33:20.102458-06
(3 filas)

C:\Users\alexg>

```

Figura 16. psql en el PostgreSQL 17 instalado en Windows (puerto 5433): las mismas siete tablas y la lectura registrada desde la aplicación.

Como prueba negativa de infraestructura detuvimos el contenedor de PostgreSQL con la aplicación corriendo: /health pasó a 503 y /lecturas mostró la página de error controlada, sin trazas ni credenciales. Al volver a arrancar la base, la aplicación se recuperó sola sin reiniciar Tomcat.

```

# Prueba negativa de infraestructura: PostgreSQL detenido (2026-09-11T18:31:27-06:00)
$ docker stop dsw-p02-huerto-db
dsw-p02-huerto-db

$ curl -s -i http://localhost:8080/web1/health
HTTP/1.1 503
Content-Type: application/json;charset=UTF-8
Content-Length: 270
Date: Sat, 12 Sep 2026 00:31:28 GMT
Connection: close

{"status":"DOWN","app":"web1 PR09 monitoreo de
huerto","db":"DOWN","dbUrl":"jdbc:postgresql://localhost:5436/huerto_db","error":"Connection to
localhost:5436 refused. Check that the hostname and port are correct and that the postmaster is
accepting TCP/IP connections."}

```

7.6 Estado de la base de datos después de las capturas

Consultado con psql usando sql/03_consultas_verificacion.sql. Se ven las siete tablas, los conteos, las lecturas con su procedencia, la alerta y las anotaciones. Las dos lecturas manuales y la anotación que empieza con '[prueba]' son exactamente las de las figuras anteriores; las dos lecturas simuladas son la semilla.

base	usuario								

huerto_db	huerto_app								
(1 row)									
table_name									

alerta									
anotacion									
lectura									
sensor									
umbral									
variable									
zona									
(7 rows)									
entidad	registros								

zona	2								
variable	4								
sensor	5								
umbral	5								
lectura	4								
alerta	1								
anotacion	2								
(7 rows)									
id	codigo	variable	zona	valor	unidad	origen	observacion	registrado_en	alerta

6	SEN-A-TEMP-01	Temperatura ambiente	Cama A	38.00	C	manual	Tarde calurosa (captura)	2026-09-12 00:31:25.370416+00	ALTA
5	SEN-A-TEMP-01	Temperatura ambiente	Cama A	27.50	C	manual	Lectura manual de prueba (captura)	2026-09-12 00:31:24.491533+00	
1	SEN-A-TEMP-01	Temperatura ambiente	Cama A	24.50	C	simulado	Carga inicial del laboratorio	2026-09-12 00:24:51.395115+00	
2	SEN-A-HSUE-01	Humedad de suelo	Cama A	55.00	%	simulado	Carga inicial del laboratorio	2026-09-12 00:24:51.395115+00	
(4 rows)									
id	codigo	nivel	mensaje				atendida	creada_en	

2	SEN-A-TEMP-01	ALTA	Temperatura ambiente en Cama A: 38 C por encima del maximo 32.00 C.				f	2026-09-12 00:31:25.370416+00	
(1 row)									
id	zona	autor_rol	texto				lectura_id	creada_en	

3	Cama A	OBSERVADOR	[prueba] Hojas con manchas en la cama A (captura)	2026-09-12 00:31:25.979371+00
1	Cama A	RESPONSABLE	Sensores instalados y calibrados con datos ficticios de laboratorio.	2026-09-12 00:24:51.395115+00

(2 rows)

8. Problemas que tuvimos y cómo los resolvimos

No todo salió a la primera y preferimos dejarlo registrado (la bitácora completa está en docs/bitacora.md).

El primer tropiezo fue con las capturas automáticas. El script elegía el sensor por posición en la lista y el primero resultó ser el de humedad de suelo, así que un 27.5 que pensábamos que era una temperatura en rango salió con alerta BAJA. Lo corregimos seleccionando por el código del sensor y poniendo cada opción del select en una sola línea del HTML para poder leer su id desde los scripts.

El segundo fue una condición de carrera con Tomcat. El script copiaba el WAR y consultaba la página de inmediato; Tomcat tarda unos segundos en replegar y volver a desplegar el contexto, y en ese lapso obtuvimos un 404 y hasta un 500 que venía del propio Tomcat, no de nuestra aplicación. La solución fue esperar a que /health respondiera 404 después de retirar el WAR y 200 después de copiarlo. De paso agregamos un listener que anula el registro del driver JDBC al parar el contexto, porque Tomcat avisaba de una posible fuga de memoria.

El tercero fue tonto pero ilustrativo: la comprobación de persistencia buscaba una marca en la salida de psql y la consulta no incluía la columna observación, así que fallaba aunque las filas sí estaban. Bastó agregar la columna a la consulta de verificación.

El más importante fue de alcance. La primera versión del modelo tenía huerto y tipo_sensor y no tenía anotación; cuando revisamos la ficha PR09 con calma vimos que las entidades, la numeración de los requisitos y hasta la idea de 'tres roles' (que son roles del sistema, no del equipo) no coincidían. Reescribimos el esquema, agregamos la página de anotaciones y volvimos a generar toda la evidencia antes de entregar.

9. Relación con la rúbrica R02

Criterio R02	Puntos	Dónde está la evidencia
Navegación JSP/Servlet y flujo funcional	2.5	Figuras 2 a 12; respuestas HTTP en docs/evidencia/txt/07 a 14c; servlets y vistas en src/
Despliegue correcto en Tomcat	2	Secciones 6 y 7.5; txt/01_build, 03_war_contenido, 05_tomcat_log, 06_health
Persistencia en PostgreSQL	2.5	Secciones 4, 7.5 y 7.6; sql/; txt/04, 15, 17 y 18; repositorios con PreparedStatement
README, capturas y repositorio	2	README.md, docs/ (acta, F02, trazabilidad, bitácora, reporte), docs/evidencia/INDICE.md, historial de commits
Orden y limpieza técnica	1	Estructura Maven estándar, paquetes por capa, .gitignore sin target ni secretos, scripts de limpieza

10. Quién hizo qué

Nos repartimos el trabajo en tres frentes y todos revisamos lo de los demás antes de cada commit.

Juan Pablo Kuri Ricardez se encargó de la parte web: los servlets, las vistas JSP con JSTL, el empaquetado con Maven y el despliegue en Tomcat, incluida la configuración por variables de entorno.

Pedro García Padilla llevó los datos: el modelo de siete entidades, los scripts de esquema, semilla, verificación y limpieza, y los repositorios JDBC con consultas parametrizadas y la transacción de lectura más alerta.

Alejandro Pacheco Luna y Ariadna Trejo Álvarez se ocuparon de la evidencia y la documentación: el protocolo de pruebas HTTP y su automatización en verify-module.sh, las capturas con navegador, el README, la bitácora, la matriz de trazabilidad, el índice de evidencia frente a R02 y este documento. También revisaron la ficha PR09 contra el código, que fue donde salió el desajuste de la sección 8.

11. Limitaciones y pendientes

Declaramos por separado lo que sí verificamos y lo que no, como pide la guía. Todo lo de las secciones 6 y 7 quedó VERIFICADO en la máquina donde se desarrolló (Windows 11). Lo que queda como NO_VERIFICADO o PENDIENTE:

Punto	Estado	Comentario
Reproducción por otro integrante en una máquina limpia	PENDIENTE	Es el CA05; el README y el script están hechos para eso
Despliegue en Linux o macOS con setenv.sh	NO_VERIFICADO	Solo probamos Windows con setenv.bat
Tres decimales y sensor inactivo por HTTP	NO_VERIFICADO	Cubiertos por pruebas unitarias; la semilla no tiene sensores inactivos
/health sin variables DB_* definidas	NO_VERIFICADO	Probamos base detenida, no configuración ausente
Atender o desactivar alertas	PENDIENTE	Prueba mínima de la ficha que dejamos para el siguiente incremento
Autenticación y autorización por rol	PENDIENTE	Fuera de Web 1.0; el rol se declara y se persiste, pero no se verifica identidad

Otras limitaciones conocidas: usamos una conexión JDBC por petición (sin pool), las fechas se muestran en la zona horaria del servidor de base de datos, y las zonas, sensores y umbrales se cargan por SQL porque su alta desde pantalla corresponde al responsable en el incremento de JSF.

Anexo. Comandos y archivos de referencia

```
# Verificación completa (desde la raíz del repositorio)
export CATALINA_HOME=/ruta/a/apache-tomcat-9.0.115
./scripts/verify-module.sh M02

# Recorrido manual con curl
B=http://localhost:8080/web1
curl -i $B/lecturas
curl -i -X POST -d "sensorId=2&valor=25.5" $B/lecturas
curl -i -X POST -d "sensorId=2&valor=abc" $B/lecturas
curl -i -X POST -d "sensorId=2&valor=38" $B/lecturas
curl -i $B/alertas
```

```
curl -i -X POST -d "zonaId=1&autorRol=OBSERVADOR&texto=Hojas+con+manchas" $B/ anotaciones

# Estado de la base y limpieza de datos de prueba
docker exec -i dsw-p02-huerto-db psql -U huerto_app -d huerto_db <
sql/03_consultas_verificacion.sql
./scripts/cleanup.sh
```

Archivo	Contenido
README.md	Requisitos, instalación, despliegue, datos de prueba, verificación, decisiones de diseño
docs/F02-seleccion-proyecto.md	Comparación de PR09, PR03 y PR01 y plantilla publicada en F02
docs/acta-proyecto.md	Acta breve con roles funcionales, alcance, flujo y RF de la ficha
docs/modelo-datos.md	Diagrama ER y diccionario de datos
docs/requisitos-trazabilidad.md	RF → código → SQL → prueba → evidencia → R02
docs/bitacora.md	Fecha, comando, resultado, interpretación y siguiente acción de cada paso
docs/verification-report.md	Estado VERIFICADO / NO_VERIFICADO / PENDIENTE por prueba
docs/evidencia/INDICE.md	Cada captura y salida relacionada con un criterio de R02